

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Архитектура вычислительных систем»

ОТЧЕТ
к лабораторной работе №2
на тему:
**«ИССЛЕДОВАНИЕ РАБОТЫ ПРОЦЕССОРА В РЕЖИМЕ ВСЕХ ЯДЕР
БЕЗ ФУНКЦИИ МНОГОПОТОЧНОСТИ»**
БГУИР 6-05-0612-02 95

Выполнил студент группы 453504
РОМАНОВ Николай Викторович

(дата, подпись студента)

Проверил ассистент каф. информатики
КАЛИНОВСКАЯ Анастасия Александровна

(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Осуществить методом математического выполнения функции согласно варианту задания. Значение аргумента x изменяется от a до b с шагом h . Для каждого x найти значения функции $Y(x)$, суммы $S(x)$ и число итераций n , при котором достигается требуемая точность $\varepsilon = |Y(x)-S(x)|$. Результат вывести в виде таблицы. Значения a , b , h и ε вводятся с клавиатуры.

$$3. \quad S(x) = \sum_{k=0}^n \frac{\cos(\frac{k\pi}{4})}{k!} x^k, \quad Y(x) = e^{\frac{x \cos \frac{\pi}{4}}{4}} \cos(x \sin \frac{\pi}{4})$$

Рисунок 1.1 – Заданные функции

2 Осуществить написание программы на любом удобном языке программирования с вставками кода ассемблера для выполнения задачи на i количестве итерации ($i > 5000$) для получения достоверных результатов эксперимента.

3 Результатом будут графики нагрузки на все ядра процессора.

По оси X – время выполнения, по оси Y – количество итераций (i).

Intel – два графика на одно ядро, с функцией Hyper-Threading и без.

AMD – с функцией SMT и без.

ARM – одно низко производительное ядро и одно высокопроизводительное ядро.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Для выполнения задания был написан код на языке C++, который вычисляет значение функции $Y(x)$ и $S(x)$. В программе используется вставка inline-ассемблера для математического сопроцессора (FPU) для процессоров архитектуры x86 (AMD) для оптимизации вычисления факториала, умножения и суммы. FPU специально оптимизирован для работы с числами с плавающей запятой, поэтому операции на нем выполняются более эффективно.

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <iomanip>
#include <chrono>
#include <vector>
#include <thread>
#include <atomic>

using namespace std;

inline double asm_add(double a, double b) {
    double result;
    __asm__ __volatile__(
        "fldl %1 \n\t"
        "fldl %2 \n\t"
        "faddp %%st, %%st(1) \n\t"
        "fstpl %0 \n\t"
        : "=m" (result)
        : "m" (b), "m" (a)
    );
    return result;
}

inline double asm_mul(double a, double b) {
    double result;
    __asm__ __volatile__(
        "fldl %1 \n\t"
        "fldl %2 \n\t"
        "fmulp %%st, %%st(1) \n\t"
        "fstpl %0 \n\t"
        : "=m" (result)
        : "m" (b), "m" (a)
    );
    return result;
}

inline double asm_factorial(int n) {
    if(n == 0 || n == 1) return 1.0;
    double result = 1.0;

    for(int i = 2; i <= n; ++i) {
```

```

        double d_i = (double)i;
        __asm__ __volatile__(
            "fldl %1 \n\t"
            "fldl %0 \n\t"
            "fmlpl %%st, %%st(1) \n\t"
            "fstpl %0 \n\t"
            : "+m" (result)
            : "m" (d_i)
        );
    }
    return result;
}

std::atomic<long long> global_completed_iters(0);
std::atomic<bool> start_flag(false);

void worker_task(long long iters_for_this_thread, double a, double b, double h,
double eps) {

    while(!start_flag.load(std::memory_order_relaxed)){

        double pi_4 = M_PI / 4.0;

        for(long long iter = 0; iter < iters_for_this_thread; ++iter) {
            for(double x = a; x <= b; x += h) {
                double y_val = exp(x * cos(pi_4)) * cos(x * sin(pi_4));
                double s_val = 0.0;
                double current_term = 0.0;
                int k = 0;

                do {

                    double numerator = asm_mul(cos(k * pi_4), pow(x, k));
                    double denominator = asm_factorial(k);
                    current_term = numerator / denominator;
                    s_val = asm_add(s_val, current_term);
                    ++k;

                } while(abs(y_val - s_val) >= eps && k < 100);

            }

            global_completed_iters.fetch_add(1, std::memory_order_relaxed);
        }
    }

    int main() {

        double a = 0.1, b = 1.0, h = 0.1, eps = 0.0001;
        long long max_iterations = 10000;

        unsigned int num_threads = std::thread::hardware_concurrency();

        if(num_threads == 0) {
            num_threads = 16;

```

```

}

cout << "Lab 2:\nDefault values:" << endl;
cout << "a: " << a << endl;
cout << "b: " << b << endl;
cout << "h: " << h << endl;
cout << "eps: " << eps << endl;
cout << "max_iterations: " << max_iterations << endl;
cout << "num_threads: " << num_threads << endl;

ofstream csv("data.csv");
csv << "Time,Iterations\n";

cout << "Starting CPU test (SMT ON/OFF)..." << endl;

global_completed_iters = 0;

long long iters_per_thread = max_iterations / num_threads;

vector<std::thread> threads;

for(unsigned int i = 0; i < num_threads; ++i) {
    threads.push_back(std::thread(worker_task, iters_per_thread, a, b, h,
eps));
}

std::this_thread::sleep_for(std::chrono::milliseconds(1));

auto start_time = chrono::high_resolution_clock::now();

start_flag.store(true, std::memory_order_relaxed);

long long record_step = max_iterations / 100;
if(record_step == 0) {
    record_step = 1;
}

long long next_target = record_step;

while(true) {
    long long current_iters =
global_completed_iters.load(std::memory_order_relaxed);
    if(current_iters >= next_target || current_iters >= max_iterations) {
        auto current_time = chrono::high_resolution_clock::now();
        chrono::duration<double> elapsed = current_time - start_time;
        csv << elapsed.count() << "," << current_iters << "\n";
        next_target = current_iters + record_step;
    }
    if(current_iters >= (num_threads * iters_per_thread)) {
        break;
    }
}

for(auto& t : threads) {

```

```

        t.join();
    }

    csv.close();

    cout << "Succesfull create data.csv" << endl;

    return 0;
}

```

Данная лабораторная была выполнена на ОС Kali Linux, поэтому все команды будут относиться именно к этому дистрибутиву.

Перед запуском программы ее нужно было скомпилировать, поэтому я использовал команду `g++ -O0 main.cpp -o lab2 -pthread`, где `-O0` указывает не оптимизировать исходный код, т.к. это может сказаться на результате.

Данный код был запущен в 2 режимах: с функцией SMT и без нее.

Для получения достоверных метрик производительности и полного исключения влияния графического сервера, параметры загрузки ядра Linux (GRUB) были модифицированы: добавлен параметр 3, что обеспечило загрузку системы в строгом текстовом режиме (`multi-user.target`).

По умолчанию функция SMT уже включена, поэтому сразу запускаем программу, используя команду `./lab2`. По результатам выполнения построим график зависимости количества выполненных итераций от времени выполнения программы.

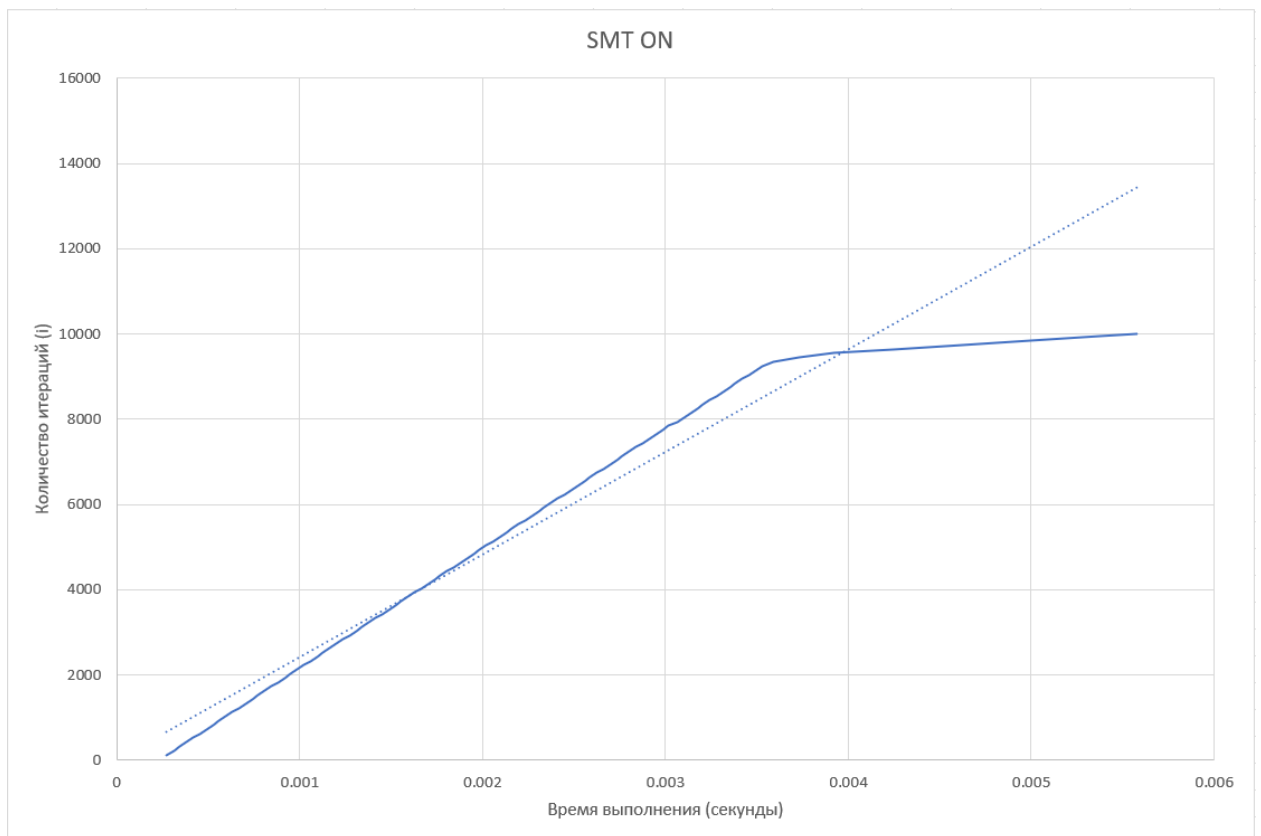


Рисунок 2.1 – График выполнения в режиме всех ядер с функцией SMT

Для запуска программы без функции SMT нужно ее принудительно отключить, сделать это можно с помощью команды `sudo tee /sys/devices/system/cpu/smt/control`, позднее с помощью этой же команды можно включить функцию SMT назад. Запускаем программу используя все ту же команду `./lab2`.

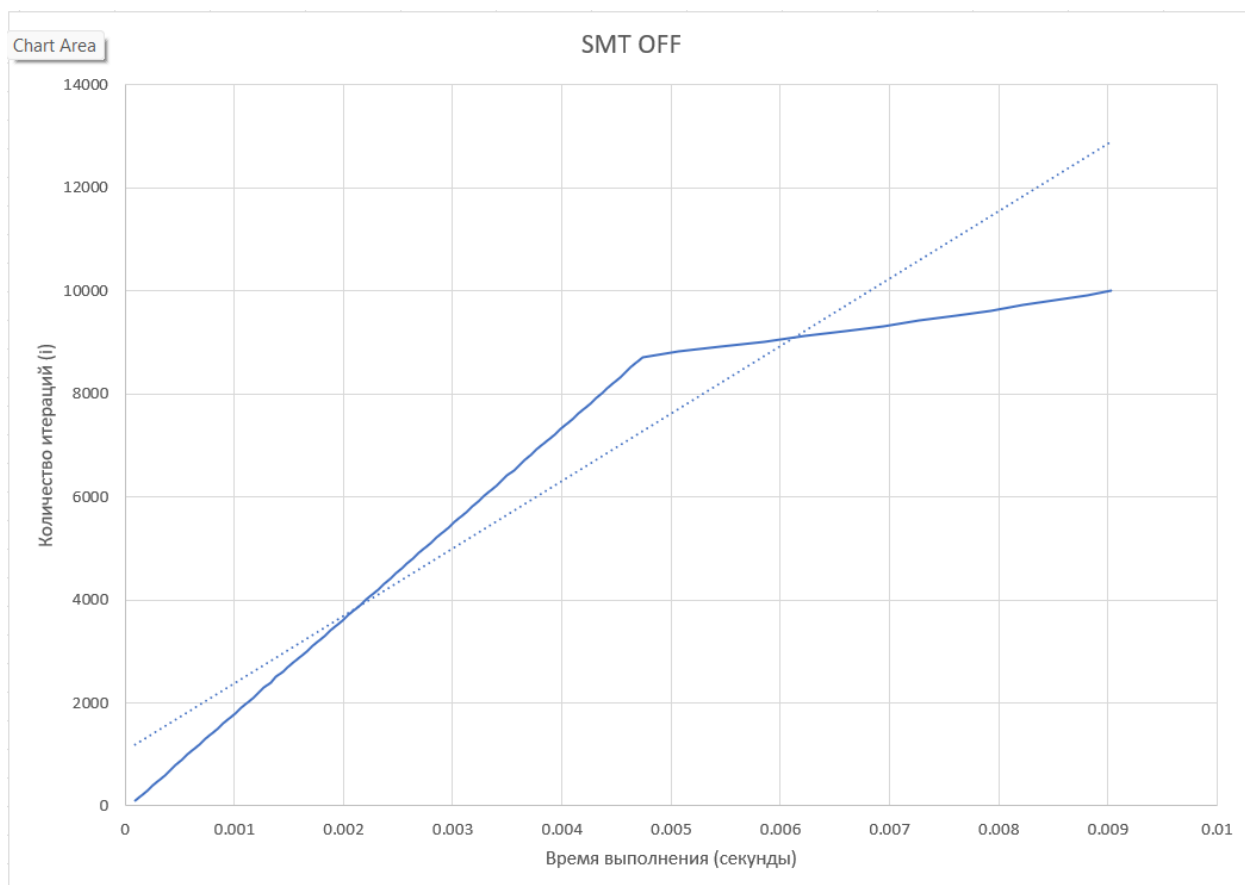


Рисунок 2.2 – График выполнения в режиме всех ядер без функции SMT

ВЫВОД

В ходе лабораторной работы был реализован многопоточный алгоритм (с использованием `std::thread`), равномерно распределяющий вычислительную нагрузку по всем доступным аппаратным потокам процессора AMD.

На графиках производительности при полной загрузке процессора наблюдается следующая картина: режим с включенной функцией SMT демонстрирует преимущество (или паритет) над режимом с отключенным SMT. Общее время выполнения 10 000 итераций с SMT ON оказывается меньше (5.5 мс), чем с SMT OFF (9 мс).

В архитектуре AMD одно физическое ядро содержит несколько математических сопроцессоров (FPU) и блоков ALU. Когда SMT выключен: На N физических ядрах работает ровно N потоков. Если поток сталкивается с задержкой (например, ждет подгрузки данных из кэша L2/L3), вычислительные конвейеры (включая FPU) простаивают. Когда SMT включен: На N физических ядрах работают $2N$ логических потоков. ОС планирует два потока на одно физическое ядро. В этом случае включается парадигма "скрытия задержек" (Latency Hiding). Если Поток №1 остановился на миллисекунду для ожидания

данных, ядро мгновенно передает простаивающий сопроцессор FPU Потoku №2.

Несмотря на то, что два потока делят один физический FPU, плотность утилизации ресурсов возрастает до максимума. В задачах с интенсивным расчетом рядов Тейлора всегда присутствуют микро-задержки (циклы while, предсказания ветвлений). SMT позволяет полезно утилизировать эти "окна простоя", что в масштабе всего процессора дает значительный прирост суммарной пропускной способности (Throughput).